Jawaban Latihan Bab 8

Queue & TicketCounterSimulation

Oleh:

MUH.ABDILLAH IRAWAN 25091397064

Mata Kuliah :

Struktur Data

PROGRAM STUDI MANAJEMEN INFORMATIKA

FAKULTAS VOKASI

UNIVERSITAS NEGERI SURABAYA

2026

1.

- **__init__ → O(n)**, karena ada perulangan untuk membuat sebanyak numAgents objek TicketAgent, sehingga waktu tumbuh linear terhadap jumlah agent.
- **run → O(m × n)**, karena ada loop sebanyak numMinutes (m), dan di setiap iterasinya memanggil tiga metode yang masing-masing melakukan scan terhadap n agent.
- **printResults → O(1)**, hanya melakukan perhitungan aritmatika sederhana dan mencetak hasilnya, tidak ada perulangan.
- **_handleArrival → O(1)**, hanya mengecek probabilitas dan melakukan satu operasi enqueue.
- **_handleBeginService → O(n)**, harus mengecek satu per satu apakah setiap agent sedang bebas, sehingga bergantung pada jumlah agent.
- **_handleEndService → O(n)**, sama seperti di atas, harus scan semua agent untuk mengecek siapa yang sudah selesai melayani.

Soal 2 — Eksekusi Manual (Enqueue)

Kode berikut melakukan iterasi nilai i dari 0 hingga 15. Setiap nilai i yang habis dibagi 3 (i % 3 == 0) akan dimasukkan ke dalam queue menggunakan operasi enqueue.

Kode: for i in range(16): if i % 3 == 0: values.enqueue(i)

Tabel penelusuran eksekusi:

i	i % 3 == 0?	Operasi	Isi Queue
0	✅ Ya	enqueue(0)	[0]
1	❌ Tidak	—	[0]
2	❌ Tidak	—	[0]
3	✅ Ya	enqueue(3)	[0, 3]
4	❌ Tidak	—	[0, 3]
5	❌ Tidak	—	[0, 3]
6	✅ Ya	enqueue(6)	[0, 3, 6]
7	❌ Tidak	—	[0, 3, 6]

i	i % 3 == 0?	Operasi	Isi Queue
8	✕ Tidak	—	[0, 3, 6]
9	☑ Ya	enqueue(9)	[0, 3, 6, 9]
10	✕ Tidak	—	[0, 3, 6, 9]
11	✕ Tidak	—	[0, 3, 6, 9]
12	☑ Ya	enqueue(12)	[0, 3, 6, 9, 12]
13	✕ Tidak	—	[0, 3, 6, 9, 12]
14	✕ Tidak	—	[0, 3, 6, 9, 12]
15	☑ Ya	enqueue(15)	[0, 3, 6, 9, 12, 15]

Hasil akhir queue (front → rear): [0, 3, 6, 9, 12, 15]

Terdapat 6 nilai yang dimasukkan ke dalam queue, yaitu semua kelipatan 3 dalam rentang 0–15.

Soal 3 — Eksekusi Manual (Enqueue + Dequeue)

Kode ini menambahkan kondisi elif i % 4 == 0 yang memicu operasi dequeue. Perlu diperhatikan bahwa kondisi elif hanya dieksekusi jika kondisi if sebelumnya tidak terpenuhi, sehingga nilai yang habis dibagi keduanya (seperti i=0 dan i=12) tetap melakukan enqueue karena if diutamakan.

Aturan: Jika i % 3 == 0 → enqueue(i). Jika tidak, dan i % 4 == 0 → dequeue().

i	i%3==0?	i%4==0?	Operasi	Isi Queue
0	☑	☑	enqueue(0) — if diutamakan	[0]
1	✕	✕	—	[0]
2	✕	✕	—	[0]
3	☑	✕	enqueue(3)	[0, 3]
4	✕	☑	dequeue() → keluar: 0	[3]
5	✕	✕	—	[3]

i	i%3==0?	i%4==0?	Operasi	Isi Queue
6	✓	✗	enqueue(6)	[3, 6]
7	✗	✗	—	[3, 6]
8	✗	✓	dequeue() → keluar: 3	[6]
9	✓	✗	enqueue(9)	[6, 9]
10	✗	✗	—	[6, 9]
11	✗	✗	—	[6, 9]
12	✓	✓	enqueue(12) — if diutamakan	[6, 9, 12]
13	✗	✗	—	[6, 9, 12]
14	✗	✗	—	[6, 9, 12]
15	✓	✗	enqueue(15)	[6, 9, 12, 15]

Hasil akhir queue (front → rear): [6, 9, 12, 15]

Dua kali operasi dequeue terjadi pada i=4 (mengeluarkan nilai 0) dan i=8 (mengeluarkan nilai 3), sehingga yang tersisa di queue hanya empat nilai terakhir.

Soal 6 — Fungsi reverseQueue

Fungsi reverseQueue dirancang untuk membalik urutan seluruh item dalam queue hanya menggunakan operasi Queue ADT, dengan bantuan sebuah stack (list Python) sebagai struktur data tambahan.

Ide utama: Queue bersifat FIFO (First In First Out), sedangkan stack bersifat LIFO (Last In First Out). Dengan memindahkan semua item dari queue ke stack, lalu memindahkan kembali dari stack ke queue, urutan item menjadi terbalik secara otomatis.

Contoh input queue (front → rear): [10, 20, 30, 40, 50]

Tahap 1 — Pindahkan semua item dari queue ke stack:

Langkah	Operasi	Stack	Queue
Awal	—	[]	[10, 20, 30, 40, 50]

Langkah	Operasi	Stack	Queue
1	dequeue() → 10	[10]	[20, 30, 40, 50]
2	dequeue() → 20	[10, 20]	[30, 40, 50]
3	dequeue() → 30	[10, 20, 30]	[40, 50]
4	dequeue() → 40	[10, 20, 30, 40]	[50]
5	dequeue() → 50	[10, 20, 30, 40, 50]	[]

Tahap 2 — Pop dari stack, enqueue kembali ke queue:

Langkah	Operasi	Stack	Queue
1	pop() → 50, enqueue(50)	[10, 20, 30, 40]	[50]
2	pop() → 40, enqueue(40)	[10, 20, 30]	[50, 40]
3	pop() → 30, enqueue(30)	[10, 20]	[50, 40, 30]
4	pop() → 20, enqueue(20)	[10]	[50, 40, 30, 20]
5	pop() → 10, enqueue(10)	[]	[50, 40, 30, 20, 10]

Hasil akhir queue (front → rear): [50, 40, 30, 20, 10]

Urutan queue berhasil dibalik. Fungsi ini hanya menggunakan operasi enqueue, dequeue, dan isEmpty dari Queue ADT, sehingga memenuhi syarat soal. Kompleksitas waktu fungsi ini adalah O(n) karena setiap item diproses tepat dua kali.

4.

Tiga metode yang perlu diimplementasikan adalah:

- **_handleArrival(curTime)** → Mengecek apakah penumpang datang pada menit ini menggunakan random.random() dibandingkan dengan _arriveProb. Jika ya, buat objek

Passenger baru dengan waktu kedatangan curTime, lalu masukkan ke _passengerQ menggunakan enqueue, dan tambahkan _numPassengers sebesar 1.

- **_handleBeginService(curTime)** → Loop melalui semua agent. Jika ada agent yang bebas (isFree()) dan queue tidak kosong, dequeue satu penumpang dari antrian, panggil agent.startService() dengan waktu berhenti = curTime + _serviceTime, lalu tambahkan selisih curTime - passenger.arrivalTime() ke _totalWaitTime.
- **_handleEndService(curTime)** → Loop melalui semua agent. Jika ada agent yang sudah selesai (isFinished(curTime)), panggil agent.stopService() untuk membebaskan agent tersebut agar bisa melayani penumpang berikutnya

5.

Perubahan yang dilakukan dari versi menit ke versi detik cukup sederhana:

1. Parameter numMinutes diganti menjadi numSeconds, dan semua referensi ke variabel tersebut ikut diganti.
2. Parameter betweenTime dan serviceTime kini diinterpretasikan dalam **detik**, bukan menit. Misalnya jika sebelumnya betweenTime = 2 berarti 2 menit, maka sekarang betweenTime = 120 berarti 120 detik.
3. Output printResults diubah labelnya dari "minutes" menjadi "seconds".

Untuk nomer 2, 3, 6 berupa koding

```python
class Queue:
    def __init__(self):
        self._data = []
    def enqueue(self, item):
        self._data.append(item)
    def dequeue(self):
        return self._data.pop(0)
    def isEmpty(self):
        return len(self._data) == 0
    def __len__(self):
        return len(self._data)
    def __repr__(self):
        return str(self._data)

# Soal 2
values = Queue()
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)

print("Soal 2 - Isi Queue:", values)
# Output: [0, 3, 6, 9, 12, 15]

#-----------------------------------------------------------------------

# Soal 3
values = Queue()
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)
    elif i % 4 == 0:
        values.dequeue()

print("Soal 3 - Isi Queue:", values)
# Output: [6, 9, 12, 15]

#-----------------------------------------------------------------------
```

```python
#-----------------------------------------------------------------

print("Soal 6 - Reverse Queue")
def reverseQueue(queue):
    stack = []
    # Keluarkan semua isi queue ke stack
    while not queue.isEmpty():
        stack.append(queue.dequeue())
    # Masukkan kembali dari stack ke queue (urutan terbalik)
    while stack:
        queue.enqueue(stack.pop())
    return queue

# Uji coba
q = Queue()
for val in [10, 20, 30, 40, 50]:
    q.enqueue(val)

print("Sebelum dibalik:", q)
reverseQueue(q)
print("Sesudah dibalik :", q)
# Sebelum: [10, 20, 30, 40, 50]
# Sesudah : [50, 40, 30, 20, 10]
```

```
Microsoft Windows [Version 10.0.19045.7184]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dell Inspiron\OneDrive\Documents\TUGAS KULIAH\struktur data pekan 12>python -u "c:\Users\Dell Inspiron\O
Soal 2 - Isi Queue: [0, 3, 6, 9, 12, 15]
Soal 3 - Isi Queue: [6, 9, 12, 15]
Soal 6 - Reverse Queue
Sebelum dibalik: [10, 20, 30, 40, 50]
Sesudah dibalik : [50, 40, 30, 20, 10]

C:\Users\Dell Inspiron\OneDrive\Documents\TUGAS KULIAH\struktur data pekan 12>
```